SPL Governance Plugins

Audit



Presented by:

OtterSec contact@osec.io

Robert Chen notdeghost@osec.io

Kevin Chow kchow@osec.io



Contents

01 Executive Summary **2**
Overview . . . 2
Key Findings . . . 2

02 Scope **3**

03 Findings **4**

04 Vulnerabilities **5**
OS-GPL-ADV-00 [med] [resolved] | Configure collection can cause potentially destructive changes 6

05 General Findings **8**
OS-GPL-SUG-00 | Enforce Governing Mint as Community Mint . . . 9
OS-GPL-SUG-01 | Make UpdateVoterWeight cumulative . . . 10
OS-GPL-SUG-02 | Enforce UpdateVoterWeight signer . . . 11

Appendices

A Program Files **12**

B Procedure **13**

C Implementation Security Checklist **14**

D Vulnerability Rating Scale **16**

© 2022 OtterSec LLC. All Rights Reserved.

01 | Executive Summary

Overview

Solana Labs engaged OtterSec to perform an assessment of `governance-program-library`.

This is an ongoing assessment, but we delivered this intermediate report on July 15th, 2022.

Key Findings

The following is a summary of the major findings in this audit.

- 4 findings total
- 0 vulnerabilities which could lead to loss of funds

© 2022 OtterSec LLC. All Rights Reserved.

02 | Scope

The source code was delivered to us in a git repository at github.com/solana-labs/governance-program-library.

We reviewed a total of 2 programs. A brief description of the programs is as follows. A full list of program files and hashes can be found in Appendix A.

Name	Description
nft-voter	A plugin to configure NFT holders as voters within a realm.
gateway	A plugin to integrate a permission layer (gateway) within a realm only for approved tokens holders for a specified gateway network.

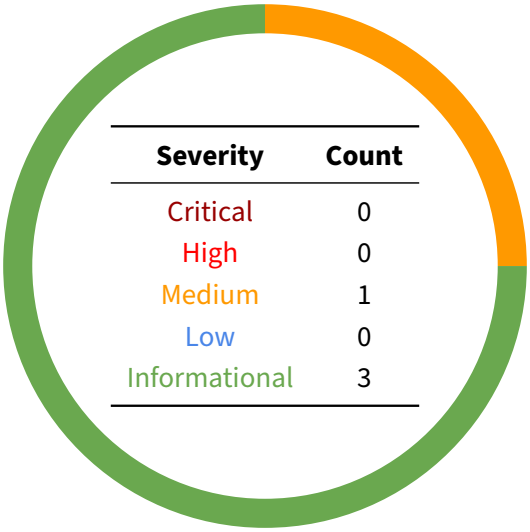
© 2022 OtterSec LLC. All Rights Reserved.

03 | Findings

Overall, we report 4 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings don't have an immediate impact but will help mitigate future vulnerabilities.

The below chart displays the findings by severity.



Severity	Count
Critical	0
High	0
Medium	1
Low	0
Informational	3

© 2022 OtterSec LLC. All Rights Reserved.

04 | Vulnerabilities

Here we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have **immediate** security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix D.

ID	Severity	Status	Description
OS-GPL-ADV-00	Medium	Resolved	`configure_collection` can cause potentially destructive changes.

© 2022 OtterSec LLC. All Rights Reserved.

OS-GPL-ADV-00 [med] [resolved] | Configure collection can cause potentially destructive changes

Description

`configure_collection` can cause potentially destructive changes to `max_voter_weight`, tipping previously untipped proposals.

As seen below, collection configuration updates the max voter weight which is used to determine if a proposal can be tipped.

Proof of Concept

1. Call `configure_collection` with collection configurations that sum to a `MaxVoterWeight` of 100.
2. Create a proposal and with a `VoterWeight` of 49.
3. Call `configure_collection` with collection configurations that sum to a `MaxVoterWeight` of 90.
4. Tip the proposal in `spl-governance`

nft-voter/src/instructions/configure_collection.rs RUST

```rust
// Update MaxVoterWeightRecord based on max voting power of the
    ↪ collections
let max_voter_weight_record = &mut
    ↪ ctx.accounts.max_voter_weight_record;

max_voter_weight_record.max_voter_weight = registrar
    .collection_configs
    .iter()
    .try_fold(0u64, |sum, cc| sum.checked_add(cc.get_max_weight()))
    .unwrap();
```

Remediation

Resolved in #56.

© 2022 OtterSec LLC. All Rights Reserved.

nft-voter/src/instructions/configure_collection.rs RUST

```rust
// Changes to the collections config can accidentally tip the
↪ scales for outstanding proposals and hence we disallow it
if realm.voting_proposal_count > 0 {
    return
↪ err!(NftVoterError::CannotConfigureCollectionWithVotingProposals);
}
```

© 2022 OtterSec LLC. All Rights Reserved.

05 | General Findings

Here we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they do represent antipatterns and could introduce a vulnerability in the future.

ID	Status	Description
OS-GPL-SUG-00		Enforce `governing_token_mint` as realm community mint
OS-GPL-SUG-01		Make UpdateVoterWeight cumulative in `nft-voter`
OS-GPL-SUG-02		Enforce `governing_token_owner` as `Signer` in UpdateVoterWeight

© 2022 OtterSec LLC. All Rights Reserved.

OS-GPL-SUG-00 | Enforce Governing Mint as Community Mint

Description

Both `nft-voter` and `gateway-civic` specify a governing mint in multiple instructions, but there is some ambiguity as a realm can have both a community and council mint. While council mints are not yet supported, we recommended enforcing the governing mint as the community mint.

gateway/src/instructions/create_registrar.rs RUST

```rust
/// Either the realm community mint or the council mint.
    /// It must match Realm.community_mint or
    ↪ Realm.config.council_mint
    ///
    /// Note: Once the Civic Pass plugin is enabled the
    ↪ governing_token_mint is used only as identity
    /// for the voting population and the tokens of that are no longer
    ↪ used
    pub governing_token_mint: Account<'info, Mint>,
```

nft-voter/src/instructions/create_registrar.rs RUST

```rust
/// Either the realm community mint or the council mint.
    /// It must match Realm.community_mint or
    ↪ Realm.config.council_mint
    ///
    /// Note: Once the NFT plugin is enabled the governing_token_mint
    ↪ is used only as identity
    /// for the voting population and the tokens of that are no longer
    ↪ used
    pub governing_token_mint: Account<'info, Mint>,
```

Remediation

The Solana Labs team communicated that they intend to support council mints on their road map in the future.

© 2022 OtterSec LLC. All Rights Reserved.

OS-GPL-SUG-01 | Make UpdateVoterWeight cumulative

Description

When stacking voter weights from NFTs in `update_voter_weight` for non-voting actions, the program is not currently iterative and limited to the number of NFTs that fit in a single transaction.

nft-voter/src/instructions/update_voter_weight_record.rs RUST

```
let mut voter_weight = 0u64;

// Ensure all nfts are unique
let mut unique_nft_mints = vec![];

for (nft_info, nft_metadata_info) in
    ↪ ctx.remaining_accounts.iter().tuples() {
    let (nft_vote_weight, _) = resolve_nft_vote_weight_and_mint(
        registrar,
        governing_token_owner,
        nft_info,
        nft_metadata_info,
        &mut unique_nft_mints,
    )?;

    voter_weight = voter_weight.checked_add(nft_vote_weight as
    ↪ u64).unwrap();
}
```

Remediation

The Solana Labs team has noted this is on the road map to support non-voting actions that require more vote weight.

© 2022 OtterSec LLC. All Rights Reserved.

OS-GPL-SUG-02 | Enforce UpdateVoterWeight signer

Description

`UpdateVoterWeightRecord` is an unprivileged instruction which allows any user to update a `VoterWeightRecord`.

nft-voter/src/instructions/update_voter_weight_record.rs RUST

```
    let voter_weight_record = &mut ctx.accounts.voter_weight_record;
    ↪  &mut Account<VoterWeightRecord>

    voter_weight_record.voter_weight = voter_weight;

    // Record is only valid as of the current slot
    voter_weight_record.voter_weight_expiry =
    ↪  Some(Clock::get()?.slot);

    // Set the action to make it specific and prevent being used for
    ↪  voting
    voter_weight_record.weight_action = Some(voter_weight_action);
    voter_weight_record.weight_action_target = None;
```

This could be used to clear the voter weight of otherwise valid `VoterWeightRecords`, making it difficult for users to vote.

Remediation

nft-voter/src/instructions/update_voter_weight_record.rs RUST

```
#[account(
        address = voter_weight_record.governing_token_owner @
    ↪  NftVoterError::InvalidTokenOwnerForVoterWeightRecord
    )]
    pub governing_token_owner: Signer<'info>,
```

© 2022 OtterSec LLC. All Rights Reserved.

A | Program Files

Below are the files in scope for this audit and their corresponding SHA256 hashes.

```
gateway
  Cargo.toml                              39a8f89132c9dfa6d32d5c476259f271
  Xargo.toml                              815f2dfb6197712a703a8e1f75b03c69
  src
    error.rs                              43575a9ae12b75e3926a3a08e83b7db1
    lib.rs                                a0f74d24b95172a72d5626e6d9e1fecf
    instructions
      configure_registrar.rs              c8e5e240ca5c4e9bb1914aa61f16b56e
      create_registrar.rs                 42cbefa01f6a44f88ebba457b62b74ea
      create_voter_weight_record.rs       d397e3e5dfc8ea0ca2859c4e8047fa9c
      mod.rs                              144b5bcb3b540ae458db7e7ecb9a0233
      update_voter_weight_record.rs       182bf206ade19cbddf33e4445935568f
    state
      generic_voter_weight.rs             3107f4dbe153a6be5882c7e06b33a222
      mod.rs                              28afafe2f655db6e74799314af2a0195
      registrar.rs                        4fc5144fe2195c27f241d04467f74b8b
      token_owner_record.rs               5a142899b13a78f7e5d34ac9287fd018
      voter_weight_record.rs              2ccfeea4eb45d0314517011281748518
    tools
      anchor.rs                           a787c048047b20edf621011f41d20951
      mod.rs                              e34e1f1ef66c74b90130db92828ad460
nft-voter
  Cargo.toml                              9998fc16c677dcfdbada2320e29d7807
  Xargo.toml                              815f2dfb6197712a703a8e1f75b03c69
  src
    error.rs                              3487093863d20d58b4c6cba41ffeae19
    lib.rs                                368bc5c23e9de7e89b047e9972b0486c
    instructions
      cast_nft_vote.rs                    6700904e3fd422d3692324280acd887f
      configure_collection.rs             8d4dd3993f85b075b6dc30e78a5ba757
      create_max_voter_weight_record.rs   999d52f098b0ac45fe8149dff9c45a02
      create_registrar.rs                 ddd1fcaf59bf04377d1f0ede443797ef
      create_voter_weight_record.rs       ffae6ec7e2f0480f8eccdc2a7089ce01
      mod.rs                              3061946c9aee0068930b9d7ed8832f9f
      relinquish_nft_vote.rs              cb232a03e60b8860c58a79ce1b5936ae
      update_voter_weight_record.rs       2b0342bbb90f7681c8d98e1ff197886c
    state
      collection_config.rs                a12936248ae7e4dd2cc9041344d0fb68
      idl_types.rs                        415d64a2cabfb0698f82f5379d0c9f0d
      max_voter_weight_record.rs          659be1678387b72e28eb3e667658e2bd
      mod.rs                              f90ecc951a0ffdfc6f5e15b7848a86fd
      nft_vote_record.rs                  89ff33decf888cda4c22841ec8fd2103
      registrar.rs                        582b038695e5da45574b3c9786cfe56f
      voter_weight_record.rs              accb2fe171600645aee43c113773836a
    tools
      anchor.rs                           a787c048047b20edf621011f41d20951
      governance.rs                       c3ae4a807acc93d9292f6260911d3cb4
      mod.rs                              9620aba59222c3ddae909ba0c9d19c15
      spl_token.rs                        e795698439f5843cb14547061f660a36
      token_metadata.rs                   f603a96651ea159ee7fb7b2a1fe74135
```

© 2022 OtterSec LLC. All Rights Reserved.

B | Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an onchain program. In other words, there is no way to steal tokens or deny service, ignoring any Solana specific quirks such as account ownership issues. An example of a design vulnerability would be an onchain oracle which could be manipulated by flash loans or large deposits.

On the other hand, auditing the implementation of the program requires a deep understanding of Solana's execution model. Some common implementation vulnerabilities include account ownership issues, arithmetic overflows, and rounding bugs. For a non-exhaustive list of security issues we check for, see Appendix C.

Implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to get a comprehensive understanding of the program first. In our audits, we always approach any target in a team of two. This allows us to share thoughts and collaborate, picking up on details that the other missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2022 OtterSec LLC. All Rights Reserved.

C | Implementation Security Checklist

Unsafe arithmetic

Integer underflows or overflows	Unconstrained input sizes could lead to integer over or underflows, causing potentially unexpected behavior. Ensure that for unchecked arithmetic, all integers are properly bounded.
Rounding	Rounding should always be done against the user to avoid potentially exploitable off-by-one vulnerabilities.
Conversions	Rust as conversions can cause truncation if the source value does not fit into the destination type. While this is not undefined behavior, such truncation could still lead to unexpected behavior by the program.

Account security

Account Ownership	Account ownership should be properly checked to avoid type confusion attacks. For Anchor, the safety of unchecked accounts should be clearly justified and immediately obvious.
Accounts	For non-Anchor programs, the type of the account should be explicitly validated to avoid type confusion attacks.
Signer Checks	Privileged operations should ensure that the operation is signed by the correct accounts.
PDA Seeds	PDA seeds are uniquely chosen to differentiate between different object classes, avoiding collision.

© 2022 OtterSec LLC. All Rights Reserved.

Input validation

Timestamps	Timestamp inputs should be properly validated against the current clock time. Timestamps which are meant to be in the future should be explicitly validated so.
Numbers	Sane limits should be put on numerical input data to mitigate the risk of unexpected over and underflows. Input data should be constrained to the smallest size type possible, and upcasted for unchecked arithmetic.
Strings	Strings should have sane size restrictions to prevent denial of service conditions
Internal State	If there is internal state, ensure that there is explicit validation on the input account's state before engaging in any state transitions. For example, only open accounts should be eligible for closing.

Miscellaneous

Libraries	Out of date libraries should not include any publicly disclosed vulnerabilities
Clippy	cargo clippy is an effective linter to detect potential anti-patterns.

© 2022 OtterSec LLC. All Rights Reserved.

D | Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings can be found in the General Findings section.

Critical	Vulnerabilities which immediately lead to loss of user funds with minimal preconditions Examples: • Misconfigured authority/token account validation • Rounding errors on token transfers
High	Vulnerabilities which could lead to loss of user funds but are potentially difficult to exploit. Examples: • Loss of funds requiring specific victim interactions • Exploitation involving high capital requirement with respect to payout
Medium	Vulnerabilities which could lead to denial of service scenarios or degraded usability. Examples: • Malicious input cause computation limit exhaustion • Forced exceptions preventing normal use
Low	Low probability vulnerabilities which could still be exploitable but require extenuating circumstances or undue risk. Examples: • Oracle manipulation with large capital requirements and multiple transactions
Informational	Best practices to mitigate future security risks. These are classified as general findings. Examples: • Explicit assertion of critical internal invariants • Improved input validation • Uncaught Rust errors (vector out of bounds indexing)

© 2022 OtterSec LLC. All Rights Reserved.